

What Can Change at Test Time?

A practical guide to test-time adaptation - and why it is different from merely giving a model more compute

Central claim. At test time, a system can change different parts of its computation - such as model parameters, normalization behavior, prompts, inputs, or temporary state - and those choices create different adaptation costs, stability risks, and memory behavior; adaptation must therefore be distinguished from simply spending more inference compute.

1. The question behind test-time adaptation

Training normally produces a set of learned parameters, often denoted by θ , and deployment then uses those parameters to make predictions. Test-time adaptation (TTA) relaxes the assumption that deployment must be completely static. When test conditions differ from training conditions, a system may use information available at test time to alter some part of its behavior. Recent TTA research is motivated especially by distribution shift: a model trained on one source distribution may encounter corruption, domain change, class imbalance, or other conditions not represented in the source data [R1, R4, R5].

The crucial educational question is therefore not simply “did adaptation happen?” It is “what was allowed to move?” EATA updates selected model parameters on reliable test samples while regularizing against forgetting [R1]. TAST adds trainable adaptation modules and uses nearest-neighbor information for pseudo-labeling [R2]. Test-Time Prompt Tuning keeps the pretrained vision-language model largely fixed while optimizing the prompt from a test sample [R3]. A broader survey organizes the literature by changes to the model, inference, normalization, sample, or prompt [R6].

2. Adaptation is not the same as more inference compute

Test-time compute and test-time adaptation are related but different axes. A model can spend more compute without changing a persistent task state: it may sample more candidate solutions, search more branches, or emit more reasoning tokens. Self-consistency, for example, samples multiple reasoning paths and aggregates the resulting answers [R10]. Tree of Thoughts explicitly explores multiple candidate “thoughts,” evaluates them, and can look ahead or backtrack [R9]. Repeated sampling has also been studied as a direct inference-scaling axis [R11].

By contrast, adaptation means that some component used by later predictions changes because of test-time information. That component might be parameters, normalization statistics, a prompt, a temporary module, or another form of state. Keeping these axes separate prevents a common conceptual error: “thinking longer” does not automatically mean “learning a new model,” and “adapting” does not automatically mean “retraining every weight.”

3. Why adaptation can fail

TTA is attractive precisely because deployment environments are uncertain, but that makes it risky. If a model adapts using its own uncertain predictions, bad pseudo-labels or noisy gradients can accumulate. EATA argues that not all test samples are equally useful and selectively updates on reliable, non-redundant samples while adding a Fisher-based regularizer to reduce forgetting [R1]. SAR studies unstable online adaptation under mixed shifts, small batches, and imbalanced streams and introduces a sharpness-aware reliable entropy-minimization procedure [R4]. DeYO shows that entropy alone can be an unreliable confidence signal in biased scenarios and proposes an additional factor-sensitive criterion [R5].

These papers point to a broader lesson: adaptation is a control problem as much as an optimization problem. A useful system needs to decide when to adapt, what to adapt, how strongly to adapt, and when to preserve the source model.

4. Context and recurrent state as a different kind of test-time change

The DataForge artifact also introduces a different research case study: BDH-CQ. Its technical report says that inference-time inputs continuously update a recurrent memory and that the query is then solved through iterative computation in a high-dimensional latent space [R8]. That behavior should not be collapsed into the classical distribution-shift TTA literature. It is better viewed as an example of test-time information changing internal recurrent state while the trained model parameters remain the machinery that performs the updates.

This distinction is useful because it separates “slow” learned parameters from “fast” task- or session-specific state. The artifact uses BDH and BDH-CQ only at the evidence level supported by their primary papers [R7, R8]. It does not claim that its educational animations reveal literal BDH hidden states or that it has independently reproduced the reported benchmark results.

5. Where test-time compute fits into the same map

Once a task has been represented, a system still needs to decide how much computation to spend before committing to an answer. Snell et al. study search with process-based verifiers and adaptive test-time allocation, finding that the most effective strategy depends on prompt difficulty [R12]. Other work moves additional computation away from visible language tokens. Coconut feeds a hidden reasoning state back into a language model as the next continuous input rather than decoding every intermediate step into words [R13]. Recurrent-depth models explicitly unroll a recurrent block further at test time to scale latent computation [R14]. HRM similarly uses interacting recurrent modules at different timescales [R15].

These are not all “TTA methods.” They are included because the artifact teaches a taxonomy: architecture answers what machinery performs computation; inference strategy answers how a compute budget is spent; adaptation answers what is allowed to change because of test-time information.

6. What the learner should be able to explain

After using the artifact, a learner should be able to inspect a test-time system and ask three separate questions. First: what are the trained parameters and what temporary state exists? Second: where is additional compute being spent - tokens, parallel samples, explicit search, or latent/recurrent updates? Third: if behavior adapts, what component actually changes? Those questions are more useful than treating every method that runs after training as one category.

The strongest takeaway is deliberately modest: test-time intelligence is not one mechanism. Different systems expose different surfaces for adaptation and different places to spend computation. Those choices change memory use, latency, stability, observability, and failure modes. A scientifically honest explainer should therefore show both the mechanism and the limits, and label published evidence separately from local educational simulation.

References

- [R1] Niu et al. (2022). *Efficient Test-Time Model Adaptation without Forgetting (EATA)*. <https://arxiv.org/abs/2204.02610>
- [R2] Jang, Chung & Chung (2022). *Test-Time Adaptation via Self-Training with Nearest Neighbor Information (TAST)*. <https://arxiv.org/abs/2207.10792>
- [R3] Shu et al. (2022). *Test-Time Prompt Tuning for Zero-Shot Generalization in Vision-Language Models (TPT)*. <https://arxiv.org/abs/2209.07511>

- [R4] Niu et al. (2023). *Towards Stable Test-Time Adaptation in Dynamic Wild World (SAR)*. <https://arxiv.org/abs/2302.12400>
- [R5] Lee et al. (2024). *Entropy is not Enough for Test-Time Adaptation: From the Perspective of Disentangled Factors (DeYO)*. <https://arxiv.org/abs/2403.07366>
- [R6] Xiao & Snoek (2024). *Beyond Model Adaptation at Test Time: A Survey*. <https://arxiv.org/abs/2411.03687>
- [R7] Kosowski et al. (2025). *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain*. <https://arxiv.org/abs/2509.26507>
- [R8] Engdahl et al. (2026). *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning*. <https://arxiv.org/abs/2608.09888>
- [R9] Yao et al. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. <https://arxiv.org/abs/2305.10601>
- [R10] Wang et al. (2022/2023). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. <https://arxiv.org/abs/2203.11171>
- [R11] Brown et al. (2024). *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling*. <https://arxiv.org/abs/2407.21787>
- [R12] Snell et al. (2024). *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*. <https://arxiv.org/abs/2408.03314>
- [R13] Hao et al. (2024/2026). *Training Large Language Models to Reason in a Continuous Latent Space (Coconut)*. <https://arxiv.org/abs/2412.06769>
- [R14] Geiping et al. (2025). *Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach*. <https://arxiv.org/abs/2502.05171>
- [R15] Wang et al. (2025). *Hierarchical Reasoning Model*. <https://arxiv.org/abs/2506.21734>